

Memory Management Simulator

Overview

This project involves designing and implementing a Memory Management Simulator that demonstrates how an operating system manages physical memory and CPU caches. The simulator helps in understanding key concepts such as dynamic memory allocation, memory fragmentation, and cache replacement policies.

The main objective is to model realistic memory behavior in a simplified and educational manner.

Memory Layout and Assumptions

Physical Memory: Simulated as a contiguous block using `std::vector<char>`, allowing byte-level access.

Memory Unit Size: Each element in memory represents 1 byte, ensuring fine-grained allocation.

Addressing Scheme: A 0-based physical address space is used, ranging from 0 to `MemorySize-1`.

Memory Alignment: As per project, **strict memory alignment (e.g., word-aligned access) is not enforced**. The simulator allows unaligned accesses at any byte boundary.

Memory Allocation Strategies

Memory is managed using an explicit linked list of memory blocks, where each block stores:

- Start address
- Block size
- Allocation status (Free or Used)

The following allocation strategies are implemented:

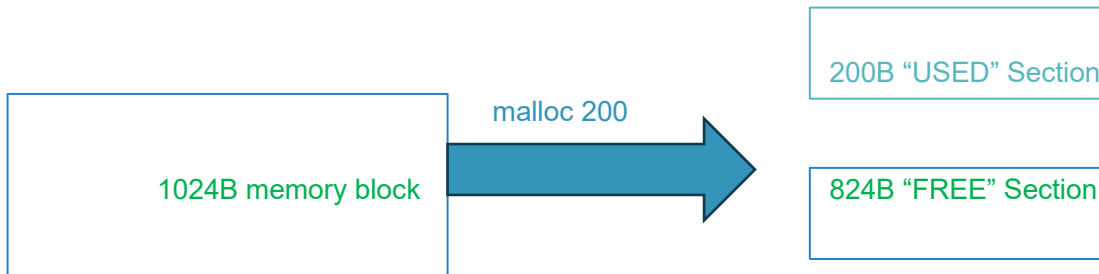
- First Fit
Allocates the first free block that is large enough to satisfy the request.
- Best Fit
Searches the entire list to find the smallest free block that can accommodate the request, reducing wasted space.
- Worst Fit
Selects the largest available free block, leaving larger remaining free segments.

Block Management

- Splitting
If a free block is larger than the requested size, it is split into:
 - One Used block of requested size
 - One Free block with remaining space

- Coalescing
When a block is freed, adjacent free blocks are merged to minimize external fragmentation.
-

Memory Allocation & Splitting



Cache Hierarchy and Replacement Policy

The simulator also models a two-level CPU cache system to mimic memory access behavior.

- Cache Levels: L1 and L2
- Configurability: Cache size and associativity can be adjusted

Cache Design

- L1 Cache: Direct-Mapped
- L2 Cache: 2-Way Set-Associative

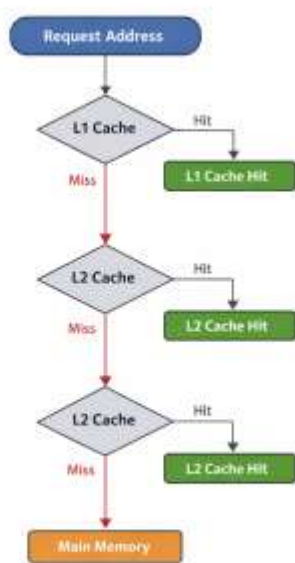
Replacement Policy

- FIFO (First-In, First-Out)
Each cache set maintains a circular pointer to track the oldest entry for eviction.

Access Flow

1. Memory request checks L1
 2. On a miss, L2 is checked
 3. If both miss, data is fetched from main memory
-

Cache Lookup Flow



Performance Metrics and Statistics

The simulator performs real-time calculations to provide performance insights during execution:

- **Memory Utilization:** The percentage of total physical memory currently marked as "Used".
- **External Fragmentation:** Calculated as the ratio of unusable free memory segments to the total free memory available.
- **Internal Fragmentation:** Consistently **0%** in this simulation. This is because memory is byte-addressable and does not utilize fixed-size blocks or paging structures that typically lead to internal waste.
- **Cache Hit Ratio:** The ratio of successful cache hits to the total number of memory accesses, tracked for both L1 and L2 levels.

Limitations and Simplifications

The current implementation focuses on the core mandatory features of physical memory management and caching. Several advanced features were omitted as per the project's scope:

- **Virtual Memory & Paging:** Address translation via Page Tables and Page Replacement policies were not implemented.
- **Cache Write Policy:** The simulator treats all memory accesses as generic loads. **Write-specific logic** (Write-Back vs. Write-Through) and **Dirty Bits** are not implemented, simplifying the cache to a Read-Only/Generic Access model.

- **Buddy Allocation:** The system uses a standard Linked List-based allocator (First/Best/Worst Fit) rather than a power-of-two Buddy System.
- **Disk Latency:** Because paging is not present, there is no symbolic modeling of disk access speeds.